

Laboratori de GIA-ABIA

Sessió 3: Cerca heurística, experimentació

En aquesta sessió anem a modificar els problemes que hem estat creant per a utilitzar algoritmes de cerca heurística.

Funcions f , g , i h

En el context de la cerca heurística, la funció $f(n)$ representa el cost estimat del camí òptim que comença a l'estat inicial, passa per n , i arriba a l'objectiu. Aquesta funció es divideix en dos components:

- $g(n)$: el cost per arribar a n des de l'inici.
- $h(n)$: el cost heurístic estimat per anar de n a l'objectiu.

La suma d'aquestes dues funcions ens dona $f(n)$, que és utilitzada per determinar l'ordre en què es visiten els nodes oberts.

En el cas d'*aima*, la funció g està implementada com `Problem.path_cost`:

```
1 def path_cost(self, c, state1, action, state2):
2     """Return the cost of a solution path that arrives at
3     state2 from state1 via action, assuming cost c to
4     get up to state1. If the problem is such that the path
5     doesn't matter, this function will only look at
6     state2. If the path does matter, it will consider c
7     and maybe state1 and action. The default method
8     costs 1 for every step in the path."""
9     return c + 1 # Valor per defecte
```

Aquesta funció retorna el cost que suma: 1) el cost c d'haver arribat a l'estat `state1` des de l'estat inicial, i 2) el cost de passar de l'estat `state1` a l'estat `state2` a través de l'operador `action`. Per defecte, el cost és 1 per cada pas en el camí.

D'altra banda, la funció heurística h pot ser proporcionada per l'usuari o definida en la subclasse `Problem` com `Problem.h`. Aquest mètode no està definit explícitament a la classe `Problem` però podeu suposar que estaria definida com:

```
1 def h(self, current_node: Node):
2     """Return the estimated heuristic cost to achieve
3     a goal state from current_state."""
4     current_state = current_node.state
5     raise NotImplementedError # Si no la definim, falla
```

Finalment, les funcions de `search.py` de cerca heurística, com ara `A*`, combinen aquestes funcions per intentar trobar el camí més curt:

```
1 def astar_search(problem, h=None, display=False):
2     h = memoize(h or problem.h, 'h')
3     return best_first_graph_search(problem,
4         lambda n: n.path_cost + h(n), display)
```

Si el nostre objecte `problem` és d'una classe amb un mètode `h` implementat per nosaltres, no cal que cridem `astar_search` amb un argument `h` i per tant pot ignorar-se o valer `None`. Aquest argument, no obstant, es pot utilitzar per sobrescriure el mètode de `problem`, per exemple per fer proves o depurar sense modificar la classe del problema.

L'argument `display`, si val `True`, mostra per consola informació sobre la progressió de l'algoritme en temps d'execució. Per espais de cerca molt grans pot ser massa informació per ser útil.

Operator comparació

Els algoritmes de cerca heurística d'*aima* utilitzen un heap per emmagatzemar els nodes de l'espai de cerca. Per tant, cal que afegim l'operador de comparació a la nostra classe de representació de l'estat, o si no rebrem el següent error:

```
1 TypeError: '<' not supported between instances of '
    ImmutableCoinsState' and 'ImmutableCoinsState'
```

Per implementar aquest operador, podem aprofitar que tenim la funció `hash` ja implementada com s'indica al guió de la Sessió 2. Aquesta funció `hash` retorna un enter i per tant podem fer una comparació directa. El valor concret de la funció de `hash` no importa, l'únic que necessitem és que els nodes es puguin ordenar al heap. Aquí teniu la funció de comparació per `ImmutableCoinsState`, tot i que us pot servir per la majoria de classes de representació de l'estat que hagueu de fer en el futur:

```
1 class ImmutableCoinsState(object):
2     ...
3     def __lt__(self, other):
4         return hash(self) < hash(other)
```

Exercicis proposats

- Rellegiu l'enunciat del problema 5 de cerca heurística i quedeu-vos amb les funcions g i h que s'hi proposen. Agafant com a base la vostra implementació, o la que us proporcionem com a sol·lució, implementeu aquestes funcions utilitzant `path_cost` i `h`.
- Proveu els següents algoritmes (els haureu d'importar de `search.py`):
 - `astar_search`
 - `uniform_cost_search`
 - `recursive_best_first_search`

Mireu el codi d'aquests algoritmes i intenteu establir una relació entre ells i els algoritmes de cerca heurística que heu vist a classe.

- De la mateixa manera que hem fet amb el problema 5, resoleu el problema 15 de cerca heurística, amb les dos funcions heurístiques que es proposen a l'enunciat.

Experimentació: Benchmarking d'Algoritmes de Cerca

En aquesta secció, ens familiaritzarem amb les eines `timeit` i `matplotlib` per realitzar benchmarks dels nostres algoritmes. L'objectiu és comparar la eficiència d'`astar_search`, `uniform_cost_search` i `recursive_best_first_search` mitjançant diverses execucions i visualitzar els resultats.

Mesurament dels temps d'execució

Per mesurar el temps d'execució d'una funció en Python, podem utilitzar la llibreria `timeit`. Primer, haurem d'importar la llibreria i després utilitzar-la per mesurar el temps d'execució dels nostres algoritmes:

```
1 from timeit import timeit
2
3 # Suposant que la funció a mesurar és astar_search
4 # i volem fer una execució :
5 time = timeit(lambda: astar_search(problem), number=1)
6 print(time) # Retorna temps en segons
```

Per fer un bon benchmarking és molt recomanable fer com a mínim 10 o 15 execucions. Com després voldrem calcular estadístiques, en comptes d'utilitzar l'argument `number` farem execucions individuals, les executarem en bucle i les guardarem en una llista:

```
1 times = [timeit(lambda: astar_search(problem), number=1)
2           for _ in range (15)]
3 print(times)
```

Aquest codi executa la funció `astar_search` un total de 15 vegades i emmagatzema els temps d'execució en la llista `times`.

Visualització dels resultats amb `matplotlib`

Suposem que hem obtingut els temps d'execució per a cada algoritme. Si volem visualitzar aquesta informació d'una manera que ens permeti fer comparacions ràpides, podem fer-ho amb boxplots utilitzant `matplotlib`.

Primer haurem d'instal·lar noves dependències:

```
1 # Previament hem d'activar l'entorn virtual amb
2 # * source venv/bin/activate, o be
3 # * venv/Scripts/activate.bat
4 pip install matplotlib
```

```
1 import matplotlib
2 matplotlib.use('agg')
3
4 import matplotlib.pyplot as plt
5 from timeit import timeit
6
7 # ...
8
9 # Obtenim els temps d'execucio
10 times_astar = [timeit(lambda: astar_search(problem), number=1)
11                for _ in range (15)]
12 times_uniform = [timeit(lambda: uniform_cost_search(problem),
13                        number=1) for _ in range (15)]
14 times_recursive = [timeit(lambda: recursive_best_first_search(
15                        problem), number=1) for _ in range (15)]
16
17 # Creem una llista amb totes les dades:
18 data_to_plot = [times_astar, times_uniform, times_recursive]
19 labels = ['astar_search', 'uniform_cost_search', '
20           recursive_best_first_search']
21
22 # Dibuixar el boxplot:
23 plt.boxplot(data_to_plot, labels=labels)
24 plt.ylabel('Temps d\'Execucio (segons)')
25 plt.title('Comparativa d\'Algoritmes amb Boxplots')
```

25 `plt.savefig('test.png')`

Aquest codi produeix un boxplot que mostra la distribució dels temps d'execució per a cada algoritme. Executeu-lo en la vostra màquina pel problema 5. Quin algoritme funciona millor i quin pitjor? Per què creieu que passa això?

Exercicis proposats

- Proveu amb variacions del problema 5 per comprovar com varien els temps d'execució. Per exemple, dupliqueu el tamany del tauler, per utilitzar 10 monedes en comptes de 5. També podeu provar amb variacions de les funcions g i h per intentar que funcioni més o menys ràpid.
- Feu una comprovació dels temps d'execució dels algoritmes amb el problema 15. Modifiqueu el codi de les gràfiques per treure una comparació entre les dos heurístiques proposades a l'enunciat. Quina heurística funciona millor? Per què?